CLAUSE BREAKDOWN

WHERE region = :region
Outer and inner WHERE both filter to a single region. The parameter is bound once and reused in both scopes.

HAVING SUM(amount) > :min_sales
HAVING filters on aggregated values *after* GROUP BY. You can't use WHERE here because the aggregate hasn't been computed yet.

WHERE sale_date >= :start_date
Applied in the subquery before aggregation, so only sales on or after the date contribute to each rep's total.

Subquery with GROUP BY
The inner query produces one row per rep with their total. The outer query joins back to get the rep's name and other profile columns.

EXECUTION ORDER

FROM / JOIN → WHERE :start_date → GROUP BY → HAVING :min_sales → WHERE :region → SELECT

Run this in Python with psycopg2 ↗

The three colored parameters — `:region` (blue), `:min_sales` (green), `:start_date` (amber) — update live as you type. A few design decisions worth noting:

**Why a subquery instead of a flat query?** The outer `JOIN` lets you pull full rep profile columns (`name`, `region`, any others) without grouping by every column — the aggregation stays clean and isolated in the inner query.

**Why HAVING and not WHERE for the amount?** `WHERE` runs before `GROUP BY`, so `SUM(amount)` doesn't exist yet when `WHERE` is evaluated. `HAVING` runs *after* aggregation, which is the only place you can filter on an aggregate result.

**Why bind `:region` twice?** Once in the subquery to restrict which rows are summed, and again in the outer `WHERE` as a safety gu since the `JOIN` could in theory surface reps from other regions if the data model allows it. In most schemas the inner filter is